

Framework Performance Benchmarking

Objective

Scope

Target Benchmarks & Success Criteria

Metrics to Track (All at P50 & P90 for Bstack and competitors)

Core Metrics (P0)

Supporting Metrics (P1)

Metrics to be instrumented

How to Run Benchmarks

Objective

Define a consistent framework to:

- Benchmark performance of Espresso, XCUITest and Maestro test runs on App Automate.
 - Identify and attribute source of delays to user/browserstack reasons
 - Establish target thresholds for performance optimisation and pick up action items to improve perf
 - Standardise metrics to track and benchmarking methodology so data is comparable across time, regions and frameworks.
-

Scope

- Frameworks: **Espresso, XCUITest, Maestro**
 - Platforms: Android (Espresso/Maestro), iOS (XCUITest/Maestro).
 - Base Metrics: Local test executions using Android studio, Xcode and Maestro without any additional capabilities
 - Components covered:
 - Pre-execution (waiting, device readiness).
 - Execution (command run time).
 - Use both:
 - Sample / synthetic sessions (controlled, reproducible).
 - Actual user sessions data for validation
-

Target Benchmarks & Success Criteria

- **North star vs local device**
 - Test execution time (session duration):

- Target: **≤ 1.1x local** for Local-Off scenarios.
 - **Bstack vs Competitor positioning**
 - Aim for **~0.5x testing time** vs competitors (e.g. LT, Sauce) for equivalent scenarios.
-

Metrics to Track (All at P50 & P90 for Bstack and competitors)

Core Metrics (P0)

1. Waiting time (queue time)

Cuts:

- Region: EUW, USE, USW, APS, APSE
- Framework: Espresso (Android) , XCUITest (IOS) , Maestro (Android + iOS)
- Reason buckets:
 - No parallel available
 - Device unavailable / tier constraint
 - Async app not ready
 - Region / device-pool constraints

2. Start time

Includes:

- Device allocation & readiness
- Enabling requested capabilities & device features
- Local / VPN tunnel setup (if enabled)
- Framework runner setup (Espresso / XCUITest / Maestro)

Cuts: Region, OS, framework

3. Total execution time (session duration)

Cuts: Region, OS, framework

	Xcuites t		Espress o		Maestro	
	Local	Bstack	Local	Bstack	Local	Bstack
Terminal/D evice readiness						
Waiting time						

Firecmd time						
Execution						

The above metrics include the data averaged across for all regions

Supporting Metrics (P1)

1. **App download + install time**
2. **Stop time** (logs, video, cleanup, device recycle)

Metrics to be instrumented

<TBD> - needs POC for each framework to understand what can be instrumented

How to Run Benchmarks

 When users run XCUITest and Espresso test suites locally using Android Studio and Xcode, both the code executors execute with SRI by default. Hence, SRI will be set to true in benchmarking. Non-SRI flows can be evaluated for performance as a P1 priority. Additionally, given that XCUITest, Espresso and Maestro are all black-box testing frameworks, we do not have insight into specific commands that the users are running within the session. Hence, we are not creating the base scripts on the basis of command usage as for Appium

1. Setup

- Choose 2 devices from Tier-1 device list.
 - Android - Samsung Galaxy S23-13.0
 - iOS - iPhone 15-17.0
- Create a base script with only default capabilities set to true.
 - Use the current sample apps and sample scripts we have for each framework and os - xcuitest, espresso and Maestro
 - In XCUITest and Espresso, ensure that the SRI is set to true
 - Do not change any other base or default capabilities
 - Loop at the existing commands in the script such that the total session duration is as per the below session durations (P90 values of duration on fwks)
 - Xcuitest: 2016
 - Espresso: 1053

- Maestro: 1198 - Android, 733 - iOS
- Toggle capabilities per scenario (LocalOn, logs, device features, etc.) - Phase 2 (post base level optimisations)

2. Execution

- Run 100+ builds per configuration for each framework × OS × region combination.

3. Instrumentation – Per Session Capture

- Core metrics:
 - waiting_time
 - terminal/device readiness
 - start/firecmd_time
 - execution_time (session duration)
- Supporting metrics:
 - app_download_time
 - app_install_time
 - test_suite_upload_time
 - app_upload_time
 - test_suite_download_time

4. Reporting and storage

- a. Capture the P90 and P50 data for the above mentioned metrics and trigger a health report email for each combination of Framework + Region + Platform + capabilities (if enabled)
- b. Store aggregated metrics in a BQ table aligned with Ideal Benchmarking doc, including:
 - i. Start, session, stop, waiting times
 - ii. Region, OS, framework, capabilities profile
 - iii. Build/session id
 - iv. P50/P90 aggregations